

Neural Network Notes

Perceptron, Gradient Reduce, Vanishing Gradients, Binary Cross-Entropy & Optimizers

1. Perceptron

The Perceptron is the simplest type of artificial neuron, introduced by Frank Rosenblatt in 1958. It is the foundation of neural networks and deep learning.

It mimics how a biological neuron works — taking inputs, processing them, and giving an output.

2. What “Gradient Reduce” Means

Step 1 — Start with a Simple Model

Tiny neural network with one weight w , predicting Pass/Fail from study hours.

Input $x = 2$ hours studied | True label $y = 1$ (Pass) | Current weight $w = 0.1$

Step 2 — Forward Pass

Prediction: $\hat{y} = w \cdot x = 0.1 \cdot 2 = 0.2$

Error (loss): $L = (y - \hat{y})^2 = (1 - 0.2)^2 = 0.64$

Step 3 — Gradient

$\partial L / \partial w = -2x(y - \hat{y}) = -2 \cdot 2 \cdot (1 - 0.2) = -3.2$

Gradient = $-3.2 \rightarrow$ means “increase weight” to reduce error.

Step 4 — Update Weight

$w := w - \alpha \cdot \text{gradient}$ ($\alpha = 0.1$)

$w = 0.1 - 0.1 \cdot (-3.2) = 0.42$

Step 5 — Next Iteration

$w = 0.42 \rightarrow \hat{y} = 0.42 \cdot 2 = 0.84$

Error: $L = (1 - 0.84)^2 = 0.0256$

Error reduced! The gradient is now smaller because we're closer to the correct answer.

What “gradient reduce” means

- As training goes on, the gradient gets smaller because the model is already close to the correct output.
- Smaller gradient $\rightarrow$ smaller weight updates.
- Eventually, gradient $\approx 0 \rightarrow$ weights stop changing $\rightarrow$ output stabilizes. This is good when the model has learned well.
- But if gradients shrink too early (like in the vanishing gradient problem), the model stops learning before reaching the best solution.

Takeaway: Gradients are not weights — they are signals. When gradients reduce, the model is getting closer to the correct output, so weights change less and predictions stabilize.

3. The Vanishing Gradient Problem

Example: a network with 5 hidden layers trying to classify handwritten digits.

Step 1 — Forward Pass

Input: image of digit “2”.

Each layer applies weights + Sigmoid activation, which squashes values between 0 and 1.

Step 2 — Backpropagation

To train, we compute error (loss); gradients flow backward layer by layer.

Each gradient is multiplied by the derivative of Sigmoid. Problem: Sigmoid's derivative is at most 0.25.

Step 3 — Shrinking Gradients

Suppose each layer's derivative ≈ 0.25 . After 5 layers: $(0.25)^5 \approx 0.00098$

The gradient reaching the first layer is almost zero.

Step 4 — What Happens

Early layers (which detect basic edges/strokes) get tiny updates and stop learning — the gradient has “vanished.”

Later layers may still adjust, but the foundation (early features) never improves.

Result: the network predicts poorly, even after many epochs.

Step 5 — Why It's Bad

The model gets stuck giving the same output (like always guessing “2” or “5”).

Training plateaus — accuracy doesn't improve, because early-layer weights stop changing.

```
Input Image -> Forward Pass -> Prediction
-> Backpropagation -> Gradients shrink ( $0.25^5 \sim 0.00098$ )
-> Early layers stop updating
-> Model stuck -> Same outputs -> Training fails
```

How We Fix It

Use ReLU instead of Sigmoid (derivative = 1 for positive inputs).

Use Batch Normalization to keep values stable.

Use Residual Connections (ResNet) so gradients can skip layers.

Use better initialization (Xavier, He) to keep gradients in a healthy range.

Takeaway

- Vanishing gradients mean the network stops learning too early — the model keeps giving the same outputs because early layers never update.
- That's why modern deep learning avoids Sigmoid/Tanh in deep networks and uses ReLU plus these tricks to keep gradients alive.

4. Binary Cross-Entropy (BCE) Loss

What it is: BCE is a loss function used in binary classification (two classes: 0 or 1). It measures how far the predicted probability is from the true label.

Formula

$$L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$$

y = true label (0 or 1) | $\hat{y}$ = predicted probability (from Sigmoid, between 0 and 1)

If $y = 1$, loss = $-\log(\hat{y})$ | If $y = 0$, loss = $-\log(1-\hat{y})$

Intuition

- Correct confident prediction → small loss.
- Wrong confident prediction → huge loss.
- BCE punishes wrong predictions more if the model was “confident but wrong.”

Case	True Label	Prediction	Loss	Result
Example 1: Correct	$y = 1$ (Pass)	$\hat{y} = 0.9$	$L = -\log(0.9) \approx 0.105$	Small loss — good prediction
Example 2: Wrong	$y = 1$ (Pass)	$\hat{y} = 0.1$	$L = -\log(0.1) \approx 2.302$	Large loss — bad prediction

5. Optimizers

An **optimizer** is an algorithm that uses the gradient (slope of the loss function) to update the weights of the network so that the loss decreases. Without optimizers, the network wouldn't know how to adjust weights to improve predictions.

Optimizer	How It Works	Pros	Cons
SGD (Stochastic Gradient Descent)	Updates weights using the gradient of each sample/batch.	Simple, widely used	Can be slow, may get stuck
Momentum	Adds “memory” of past gradients to speed up convergence.	Faster, smoother updates	Needs tuning of momentum parameter
RMSProp	Adjusts learning rate based on recent gradient magnitudes.	Good for recurrent networks	May converge to a suboptimal solution
Adam (Adaptive Moment Estimation)	Combines momentum + adaptive learning rates.	Fast, robust, most popular	Slightly more complex, may overfit
Adagrad	Adjusts learning rate per parameter (large updates for rare features).	Good for sparse data	Learning rate shrinks too much over time